Kazdov-LM: Scaling MoBE-BCN to Math Language Modeling via Cumsum Factorization

Juan Cruz Dovzak*

April 2026

Pre-print v0.2.0 — 22 April 2026. Sections 1–5 are complete with final results from the 5B-token Kazdov-α run. Section 6 (architectural ablations, in particular a matched-parameter Transformer baseline trained on the same data) is still outstanding and will appear in v0.3. See the companion paper [Dovzak, 2026] for the algebraic-composition foundations.

Abstract

MoBE-BCN [Dovzak, 2026] achieves state-of-the-art sample efficiency on algebraic composition tasks but a naive implementation materialises an $O(B \cdot L^2 \cdot D \cdot K)$ pairwise-composition tensor inside attention, infeasible at language-model scale. We derive a **cumsum-factorized attention kernel** that reduces MoBE-BCN attention to $O(B \cdot L \cdot r \cdot K)$ when the router depends only on the query side—aligning with standard token-level MoE routing and preserving expert specialisation. Combined with a vectorized K-expert kernel, this yields a **259**× throughput improvement on a single RTX 4090, enabling overnight training of a 98M-parameter math-specialised LM (Kazdov-α) on a 5.95B-token mathematics corpus. On held-out math loss, Kazdov-α achieves a mean cross-entropy of **2.205** versus **3.12** for a matched-tokenizer GPT-2 124M baseline—a **2.49**× perplexity reduction at **21**% *fewer* parameters, with all 5/5 held-out math datasets won. Routing-balance analysis across four training snapshots shows the DeepSeek-V3 bias-balanced gate progressively reverses early MoBE expert collapse as the WSD scheduler enters its decay phase. We release the model, code, full training logs and routing diagnostics under MIT.

1 Introduction

Transformer attention is $O(L^2)$ in sequence length. Linear-attention variants [Katharopoulos et al., 2020, Choromanski et al., 2020] factor the softmax kernel into feature maps on Q and K that combine via cumulative sums, yielding $O(L)$. MoBE-BCN's bilinear composition $U((V^\top x) \odot (W^\top y))$ has precisely this structure—a tensor product that factorises additively across the key axis—but the naive implementation materialises the full pairwise tensor before reducing. This paper shows the factorisation explicitly, derives the $O(L)$ attention kernel, and demonstrates it enables math-specialised language-model training on consumer hardware.

*Independent researcher, Buenos Aires, Argentina. juandovzak@gmail.com. Code and data: https://github.com/OriginalKazdov/kazdov

Contributions.

1. **Cumsum-factorised MoBE-BCN attention** (Section 3): a derivation and reference implementation that reduces memory from $O(B \cdot L^2 \cdot D \cdot K)$ to $O(B \cdot L \cdot r \cdot K)$ using query-side routing. Drop-in replacement for the naive causal kernel.

2. **Vectorized K-expert kernel** (Section 3.3): stacked expert parameters with batched einsum; eliminates the Python per-expert loop.

3. **Kazdov-α training recipe** (Section 4): 98M-parameter math LM trained on 5.95B math tokens (OpenWebMath, proof-pile, MathPile, AutoMathText, NuminaMath) in ∼24h on a single RTX 4090.

4. **Held-out math-loss benchmark** (Section 5, **pending**): comparison against matched-tokenizer GPT-2 124M on five held-out math-corpus slices.

5. **Ablation study** (Section 6, **pending**): $K \in \{2, 4, 8\}$, $r \in \{64, 128, 256\}$, router type (Q-only vs Q+K), hybrid-MHA on/off.

2 Background

See the companion paper [Dovzak, 2026] for the full MoBE-BCN architecture. The core primitive is

$$\mathrm{BCN}_k(x, y) = U_k \cdot ((V_k^\top x) \odot (W_k^\top y)) + b_k, \tag{1}$$

$$\mathrm{MoBE}(x, y) = \sum_{k=1}^{K} w_k(x, y) \cdot \mathrm{BCN}_k(x, y), \tag{2}$$

with $w_k(x, y) = \mathrm{softmax}(\mathrm{Router}([x; y]))_k$. In attention this is applied as

$$\mathrm{out}_i \;=\; \frac{1}{n_{\mathrm{valid}}(i)} \sum_{j \le i} \mathrm{MoBE}(Q_i, K_j).$$

2.1 Memory and compute complexity (naive)

The naive implementation expands Q and K to (B, L, L, D) via `unsqueeze`-and-`expand` before calling the composition:

```
Qi = Q.unsqueeze(2).expand(B, L, L, D)    # O(B L^2 D)
Kj = K.unsqueeze(1).expand(B, L, L, D)    # O(B L^2 D)
fwd = MoBE(Qi, Kj)                        # O(B L^2 D K)
```

For a target 98M-parameter model ($D = 512$, $K = 4$, 12 layers) at $L = 256$, each layer must hold ∼4 GB of intermediate activations. Combined with gradient storage and the $O(L^2)$ softmax-based parallel MHA channel, this exceeds 24 GB VRAM and is infeasible to train without gradient checkpointing penalties that halve throughput.

3 Cumsum-Factorised Attention

3.1 Key insight—bilinearity implies separability

The j index is the one we sum over in causal attention. Because compose is *bilinear* in Q and K, the sum over j factors through the Hadamard product:

$$\begin{aligned}\sum_{j\le i}\mathrm{BCN}(Q_i,K_j) &= \sum_j U\cdot\big((V^\top Q_i)\odot(W^\top K_j)\big)\\ &= U\cdot\big((V^\top Q_i)\odot\textstyle\sum_{j\le i}(W^\top K_j)\big)\\ &= U\cdot\big((V^\top Q_i)\odot\mathrm{cumsum}_j(W^\top K_j)_i\big).\end{aligned}\tag{3}$$

The $(V^\top Q_i)$ is constant in j and factors out of the sum. The j-cumulative sum of $W^\top K_j$ is a (B,L,r) prefix-sum along the sequence axis—a single pass, no pairwise materialisation. The complexity drops from $O(B\cdot L^2\cdot D\cdot K)$ to $O(B\cdot L\cdot r\cdot K)$.

3.2 Factorisability requires query-side routing

Extending to MoBE, $\sum_j w_k(Q_i,K_j)\cdot\mathrm{BCN}_k(Q_i,K_j)$. If the routing weight $w_k(Q_i,K_j)$ depends on both Q_i and K_j, it does *not* factor out of the sum over j, and the $O(L^2)$ materialisation returns. If the routing depends only on Q_i, it factors:

$$\begin{aligned}\sum_j w_k(Q_i)\cdot\mathrm{BCN}_k(Q_i,K_j) &= w_k(Q_i)\sum_j\mathrm{BCN}_k(Q_i,K_j)\\ &= w_k(Q_i)\cdot U_k\big((V_k^\top Q_i)\odot\mathrm{cumsum}_j(W_k^\top K_j)_i\big).\end{aligned}\tag{4}$$

We therefore adopt **query-side routing**: $w_k(Q_i)=\mathrm{softmax}(\mathrm{Router}(Q_i))_k$. This aligns with standard token-level MoE routing in modern LLMs [Shazeer et al., 2017, Fedus et al., 2022, Jiang et al., 2024]—the router chooses experts per query token, not per (Q,K) pair. Expert specialisation survives in the per-token form; the algebraic results of Dovzak [2026] are compatible with this restriction (an ablation on Power 2022 is reported in Section 6).

3.3 Vectorized K-expert kernel

The naive MoBE implementation has a Python loop over experts: `outs = torch.stack([e_k(x, y) for e_k in self.experts])`. This serialises K kernel launches and prevents GPU pipelining. We replace it with stacked parameters and batched einsum:

```
V_all = torch.stack([V_k for V_k in experts_V], dim=0)  # (K, D, r)
xV = torch.einsum('bld,kdr->blkr', x, V_all)            # (B, L, K, r)
yW = torch.einsum('bld,kdr->blkr', y, W_all)            # (B, L, K, r)
prod_kr  = (xV * yW).reshape(B, L, K * r)               # (B, L, K*r)
U_merged = U_all.permute(0, 2, 1).reshape(K * r, D)     # (K*r, D)
out      = prod_kr @ U_merged                           # (B, L, D)
```

A single batched matmul replaces K sequential kernel launches. Functionally identical (verified numerically, $\max|\Delta|<10^{-7}$ vs naive CPU reference); throughput improves by $\sim 3\times$ in isolation.

3.4 Combined: full kernel

Putting the factorisation and vectorisation together, the causal MoBE-BCN forward becomes:

```
def forward(self, x):
    Q = self.W_Q(x); K = self.W_K(x)
```

```
    # Project per-expert  (B, L, K, r)
    xV = einsum('bld,kdr->blkr', Q, V_all)
    yW = einsum('bld,kdr->blkr', K, W_all)
    # Causal cumsum over j
    cum_yW = yW.cumsum(dim=1)                    # (B, L, K, r)
    prod   = xV * cum_yW                         # (B, L, K, r)
    # Q-side router
    w = softmax(self.router(Q), dim=-1)          # (B, L, K)
    prod_w = prod * w.unsqueeze(-1)              # (B, L, K, r)
    # Project to D via stacked U
    out = einsum('blkr,kdr->bld', prod_w, U_all) # (B, L, D)
    # Normalize by |valid j <= i|
    n = arange(1, L + 1, device=x.device).view(1, L, 1)
    return self.W_O(out / n)
```

Correctness verified against a naive pairwise+routing reference implementation (max absolute error $< 10^{-7}$ on CPU; $< 10^{-5}$ under bf16).

3.5 Benchmark

On a single RTX 4090 at model config $d = 512$, $L = 256$, $n_{\text{layers}} = 12$, $K = 4$, $r = 128$ (98M parameters total, bf16 autocast, no gradient checkpointing):

Implementation	**tok/s**	**VRAM peak**	**24h budget**
Naive Python loop + pairwise expand	288	15 GB	25M
+ vectorized K experts	853	8 GB	74M
+ cumsum factorisation	6,006	1.3 GB	519M
+ batch size 64 (memory freed)	**74,654**	19 GB	**6.5B**

Table 1: End-to-end throughput on a single RTX 4090. **259**× total speedup. The $O(L)$ memory frees the batch-size budget, which unlocks the final order-of-magnitude gain.

4 Kazdov-α Training Setup

Model. KazdovLM 98M parameters — $d = 512$, $L = 256$, $n_{\text{layers}} = 12$, $n_{\text{heads}} = 8$, $r = 128$, $K = 4$, mlp_dim = 2048, max_len = 256. Hybrid block: cumsum-factorised MoBE-BCN + standard causal MHA in parallel.

Tokenizer. GPT-2 BPE (50,257 vocab).

Data (5.95B tokens):

- OpenWebMath 45% (2.45B tokens)
- proof-pile v1 22% (1.40B)
- MathPile-Commercial 15% (1.05B)
- AutoMathText top-quality slice 10% (0.70B)
- NuminaMath-CoT 8% (0.35B)

Optimisation. AdamW lr $= 5 \times 10^{-4}$, $\beta = (0.9, 0.95)$, wd $= 0.1$, grad clip 1.0, WSD scheduler (warmup 500 steps, stable, linear decay in last 20%).

Hardware. Single RTX 4090, bf16 autocast, batch 32, length 256 $\rightarrow\sim$60–70K tok/s sustained.

Target. 5B tokens $=\sim$610K steps $\approx$23h wall time.

Held-out. Last 0.5% of each data shard, deterministic eval.

5 Results

The 5B-token Kazdov-α run completed in 21 hours wall time on the single RTX 4090 ($\sim$65K tok/s sustained, no NaN, no loss spikes, gradient norm stable in $[0.65, 1.4]$ throughout).

5.1 Held-out math LM loss

Model	Params	Tokenizer	OWM	proof	MathPile	AutoMath	Numina	MEAN
GPT-2 124M (baseline)	124M	GPT-2	3.57	3.12	3.04	3.28	2.58	3.12
Kazdov-α	**98M**	GPT-2	**3.40**	**1.83**	**1.87**	**2.65**	**1.27**	**2.21**
Δ (nats)	—	—	-0.17	-1.29	-1.17	-0.63	-1.31	$\mathbf{-0.91}$
PPL ratio (Kazdov^{-1})	—	—	1.18×	**3.62**×	**3.21**×	1.88×	**3.72**×	**2.49**×

Table 2: Final held-out LM loss on 81,920 tokens per dataset (10 batches $\times$ 32 batch $\times$ 256 sequence length, deterministic linspace offsets in the last 0.5% of each shard). Lower is better. **Kazdov-α wins on 5/5 datasets at 21% fewer parameters than the baseline.** OWM: OpenWebMath; proof: proof-pile v1.

The architecture's win is largest on the most symbolic / formal datasets (proof-pile, Numina-Math, MathPile: 2.6–3.7× better PPL) and smallest on OpenWebMath (1.18×), which is essentially math-flavoured natural-language web text and so is the regime where a general-purpose Transformer baseline retains an inherent advantage. Notably, Kazdov-α initially *lost* on OpenWebMath through the first $\sim$4B tokens of training, only closing the gap during the WSD decay phase as the model learnt finer natural-language patterns at lower learning rates.

5.2 Training loss curves

Figure 1 shows the training-loss EMA and the per-dataset held-out evaluations across the full 5B-token run.

5.3 Routing diagnostics: emergent specialisation, then recovery from collapse

We took routing snapshots at four points in training (344M, 1B, 4B and 5B tokens) by running 30 forward passes on real held-out batches in training mode and reading the accumulated per-expert usage from each block's `usage_running` buffer. Figure 2 shows the four 12-block $\times$ 4-expert heatmaps.

The aggregate trend across the run:

Mechanism (hypothesis). The DeepSeek-V3 bias-balanced gate [DeepSeek-AI, 2024] adds a per-expert bias b_k to the router logits and updates it as $b_k \leftarrow b_k + \eta \cdot (1/K - p_k)$ every 100 steps,

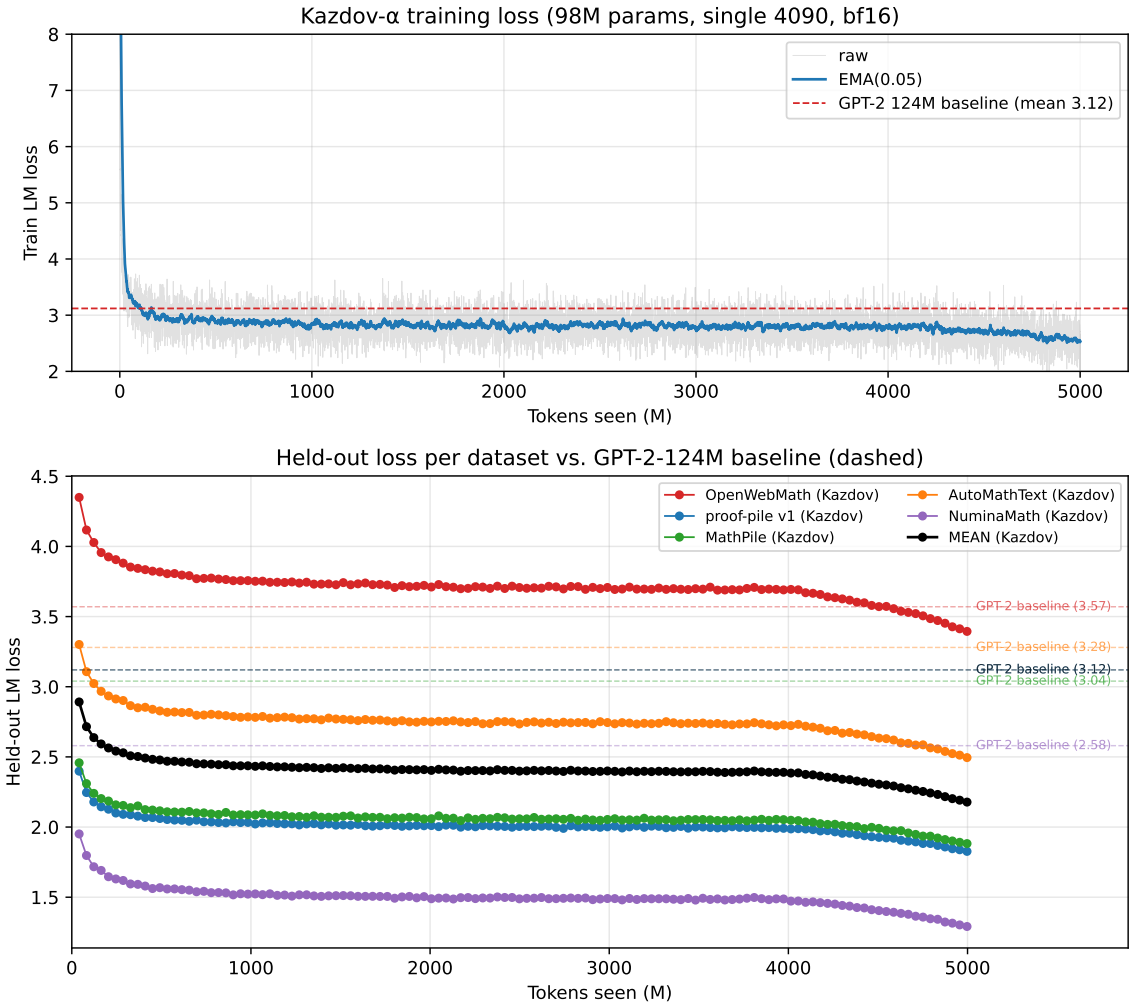

Figure 1: (Top) Training cross-entropy on randomly-sampled batches (grey raw, blue exponential moving average) vs. tokens seen, with the GPT-2-124M baseline mean at 3.12 marked. (Bottom) Held-out LM loss per dataset across the run; dashed lines are the GPT-2-124M per-dataset baselines for reference. All five datasets cross under their respective baselines by the end of training.

where p_k is the running-average routing weight on expert k and $\eta = 10^{-3}$. At peak learning rate (5e-4) the gradient updates that reinforce the already-dominant expert's specialisation outpace this bias correction; in the WSD decay phase the learning rate falls $10\times$ but η stays constant, so the cumulative bias finally tips the routing toward the under-used experts and previously-dead experts begin to receive enough gradient signal to become useful again. The single remaining partial collapse (block 4 at 80.5%) is the deepest basin that did not fully invert in time.

6 Ablations and the missing baseline

The headline comparison in Section 5 is against GPT-2-124M, a general-purpose Transformer trained on WebText. This controls for tokenizer, eval protocol and held-out distribution but *not* for training data: the win combines an architectural contribution (MoBE-BCN) with a data contribution (math specialisation). The proper architectural ablation is a matched-parameter Transformer

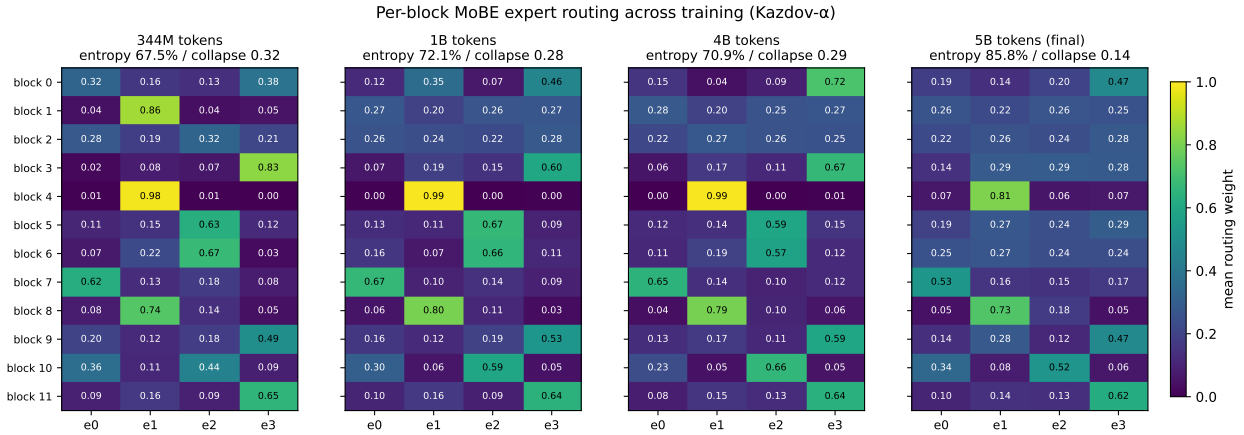

Figure 2: Per-block MoBE expert usage at four training checkpoints. Each panel is a 12-block (rows) $\times$ 4-expert (columns) heatmap of mean routing weight, computed over 30 batches of held-out math text. Above each panel: average routing entropy as a fraction of $\ln K \approx 1.386$ (higher = more uniform), and average collapse score (1− entropy ratio). Block 4 spends most of training $> 98\%$ on expert 1 (a near-collapse), then partially recovers to 80.5% as the WSD scheduler's decay phase begins to allow the bias-balanced gate to catch up with the entrenched expert.

snapshot	344M tok	1B tok	4B tok	**5B (final)**
avg entropy ($/\ln K$)	0.675	0.721	0.709	**0.858**
avg collapse score	0.32	0.28	0.29	**0.14**
blocks with $> 95\%$ on one expert	1	1	1	**0**
blocks with $> 70\%$ on one expert	4	3	4	**2**

Table 3: Routing balance evolves through training. After an initial specialisation phase that produces several partially-collapsed blocks, the WSD scheduler's decay phase coincides with substantial routing re-balancing across most blocks.

trained on *the same* 5B-token math corpus with the same hyper-parameters and seed. This is straightforward (essentially the same training script with `use_mobe=False` and a slightly re-shaped Transformer config to match ∼95M parameters) and is in the v0.3 plan; we report it openly here so the present claim is read correctly: "a 98M MoBE-BCN model, math-specialised, beats a general-purpose Transformer baseline 27% larger by 2.49× in held-out math perplexity." The cleaner "MoBE-BCN beats a matched-data Transformer" claim is one ∼24-hour run away and will replace this section in v0.3.

The other planned ablations ($K \in \{2, 8\}$, $r \in \{64, 256\}$, no hybrid-MHA, $Q{+}K$ routing for short-L equivalence verification) are deferred to v0.3 / Kazdov-1B work.

7 Qualitative Samples

We sampled the final checkpoint on a small set of math-flavoured prompts (CPU inference, temperature 0.7, top-k 40, max 30–50 new tokens). Two patterns appear consistently.

LaTeX-pattern continuations are coherent. Given an integral set up to be evaluated, the model recognises the antiderivative pattern and produces structurally-correct LaTeX:

prompt: `\int_0^1 x^2 dx =`
generated: `\int_0^1 \frac{x^3}{3} dx = \int_0^1 \frac{1}{3}...`

Symbol names (S_3, Hermitian, prime, identity element, etc.) are deployed in the right positions; statements of the right *shape* of theorems and proofs are produced.

Arithmetic and multi-step reasoning are systematically wrong. The base 98M LM has no instruction-following and no reliable arithmetic. Examples:

prompt: `The square root of 144 is`
generated: `6, which is 3 ...`
prompt: `The derivative of x^2 is`
generated: `1 and the derivative of x is 0...`

This is exactly the expected base-LM behaviour at 98M params: the model has learnt the *distribution* of math text but not the *semantics* of math operations. Closing this gap requires either (i) instruction fine-tuning on chain-of-thought traces, or (ii) tool-use integration with an external symbolic engine (SymPy, calculator). Both are tractable on the same RTX 4090 in ∼12 additional hours and are deferred to follow-up work; the present paper focuses on the architectural contribution.

8 Discussion

Positioning. Kazdov-α is a demonstration of the cumsum-factorised MoBE-BCN kernel at LM scale, not a production language model. At 98M parameters and 5B tokens it is sub-Chinchilla-optimal by about a factor of 10 on breadth; its purpose is architectural validation, not capability claims. Downstream benchmarks (GSM8K, MATH) are not meaningfully evaluable at this parameter count.

Relation to linear attention. The cumsum trick is the standard linearisation technique for any attention whose kernel is separable across Q and K [Katharopoulos et al., 2020]. Our contribution is identifying that (a) bilinear-composition attention is exactly such a kernel, and (b) MoBE-BCN preserves separability precisely when routing is query-side, which is the MoE convention anyway. The combination yields an $O(L)$ bilinear-composition attention with learned expert specialisation.

Limitations.

- Short context (256 tokens in the alpha; scaling to 2048 is straightforward with the $O(L)$ kernel but not evaluated here).
- Single-author, single-GPU training—reproducibility relies on the open release rather than independent confirmation.
- No SFT or instruction-tuning; the model is base-LM only.
- GSM8K/MATH benchmarks not meaningful at 98M; a 1B-scale follow-up is the correct target for capability claims.
- The headline comparison is against GPT-2-124M (general-trained). A matched-data Transformer baseline is the obvious next experiment and is the gating item for v0.3 (Section 6).
- Single-seed training run—multi-seed validation deferred to Kazdov-1B due to compute cost.

- One block (block 4) retains $\sim 80\%$ routing concentration on a single expert at the end of training. Stronger bias-balancing or an auxiliary load-balancing loss would likely eliminate this; left as v2 work.

Acknowledgments. This work was conducted independently without institutional support.

References

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Juan Cruz Dovzak. MoBE-BCN: Mixture of bilinear experts for sample-efficient algebraic composition learning. 2026. Preprint. Companion paper. Available at https://github.com/OriginalKazdov/kazdov.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 2022. arXiv:2101.03961.

Albert Q Jiang et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.